SQL Basics: Answer Sheet

Solutions to the ten questions in the banking use case sheet. Each query returns the expected output shown there. Syntax is MySQL and PostgreSQL; differences are noted where they matter.

Segment 1: Clauses and table structure altering

Q1. WHERE + ORDER BY

```
SELECT txn_id, account_id, txn_date, amount
FROM transactions
WHERE txn_type = 'DEBIT'
  AND channel  = 'UPI'
  AND amount   > 10000
  AND txn_date BETWEEN '2026-09-01' AND '2026-09-30'
ORDER BY amount DESC;
```

Key idea: WHERE filters rows before anything else runs; ORDER BY … DESC sorts largest first.

Q2. GROUP BY + HAVING

```
SELECT branch_city,
       COUNT(*)     AS num_accounts,
       SUM(balance) AS total_balance
FROM accounts
GROUP BY branch_city
HAVING SUM(balance) > 100000
ORDER BY total_balance DESC;
```

Key idea: WHERE cannot filter on an aggregate; HAVING filters groups after GROUP BY.

Q3. JOIN + CASE + LIMIT

```
SELECT a.customer_name,
       COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN  t.amount
                         WHEN t.txn_type = 'DEBIT'  THEN -t.amount
                    END), 0) AS net_flow
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
GROUP BY a.customer_name
ORDER BY net_flow DESC
LIMIT 3;
```

Key idea: CASE turns debits negative so one SUM gives the net. COALESCE turns Sneha Iyer's NULL into 0, so the result is the same on MySQL and PostgreSQL. SQL Server uses `SELECT TOP 3` instead of `LIMIT 3`.

Q4. ALTER TABLE: rename and modify a column

```
ALTER TABLE accounts RENAME COLUMN branch_city TO branch_location;

-- MySQL (repeat NOT NULL, or MODIFY drops it)
ALTER TABLE accounts MODIFY customer_name VARCHAR(150) NOT NULL;

-- PostgreSQL
ALTER TABLE accounts ALTER COLUMN customer_name TYPE VARCHAR(150);

-- Check the new structure
DESCRIBE accounts;      -- MySQL
-- \d accounts          -- PostgreSQL (psql)
```

Key idea: ALTER changes the definition, not the data; all 6 rows stay as they were.

Q5. ALTER TABLE: add a column and a constraint

```
ALTER TABLE accounts ADD COLUMN email VARCHAR(100);

ALTER TABLE transactions
ADD CONSTRAINT chk_amount_positive CHECK (amount > 0);

-- Test: this insert must fail
INSERT INTO transactions
VALUES (11, 101, '2026-09-20', 'DEBIT', -500.00, 'UPI');
```

Key idea: a new column is NULL for existing rows. CHECK rejects only values that evaluate to false, so the existing NULL amount is allowed. MySQL enforces CHECK from version 8.0.16.

Segment 2: Data profiling

These run after Segment 1, so the city column is `branch_location`.

Q6. Row counts and distinct values

```
SELECT
  (SELECT COUNT(*) FROM accounts)                       AS accounts_rows,
  (SELECT COUNT(*) FROM transactions)                   AS
transactions_rows,
  (SELECT COUNT(DISTINCT account_type) FROM accounts)   AS
distinct_account_types,
  (SELECT COUNT(DISTINCT branch_location) FROM accounts) AS
distinct_branch_locations;
```

Key idea: COUNT(*) counts every row; COUNT(DISTINCT col) counts unique non-NULL values.

Q7. Missing values

```
SELECT
  SUM(CASE WHEN account_type IS NULL THEN 1 ELSE 0 END) AS null_account_type,
  SUM(CASE WHEN balance      IS NULL THEN 1 ELSE 0 END) AS null_balance
FROM accounts;

SELECT
  SUM(CASE WHEN txn_date IS NULL THEN 1 ELSE 0 END) AS null_txn_date,
  SUM(CASE WHEN amount   IS NULL THEN 1 ELSE 0 END) AS null_amount,
  SUM(CASE WHEN channel  IS NULL THEN 1 ELSE 0 END) AS null_channel
FROM transactions;
```

Key idea: test NULL with IS NULL, never `= NULL`. A shorter form is `COUNT(*) - COUNT(col)`, since COUNT(col) skips NULLs.

Q8. Summary statistics

```
SELECT txn_type,
       COUNT(*)              AS txn_count,
       MIN(amount)           AS min_amount,
       MAX(amount)           AS max_amount,
       ROUND(AVG(amount), 2) AS avg_amount,
       SUM(amount)           AS total_amount
FROM transactions
GROUP BY txn_type
ORDER BY txn_type;
```

Key idea: aggregate functions ignore NULLs, so the DEBIT average divides by 6, not 7.

Q9. Duplicate detection

```
SELECT account_id, txn_date, txn_type, amount,
       COUNT(*) AS occurrences
FROM transactions
GROUP BY account_id, txn_date, txn_type, amount
HAVING COUNT(*) > 1;
```

Key idea: group by every column that defines "the same transaction" (leave out txn_id), then keep groups with more than one row.

Q10. Accounts with no transactions

```
SELECT a.account_id, a.customer_name
FROM accounts a
LEFT JOIN transactions t ON a.account_id = t.account_id
WHERE t.txn_id IS NULL;

-- Alternative
SELECT account_id, customer_name
FROM accounts a
WHERE NOT EXISTS (
  SELECT 1 FROM transactions t WHERE t.account_id = a.account_id
);
```

Key idea: a LEFT JOIN keeps every account and fills unmatched transaction columns with NULL; filtering on that NULL leaves only accounts with no match.